SKILL WEEK-3

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

CODE:

navaneeth@Navaneeth: ~

GNU nano 8.7.1 shell.c

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];

    while (1)
    {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

OUTPUT:

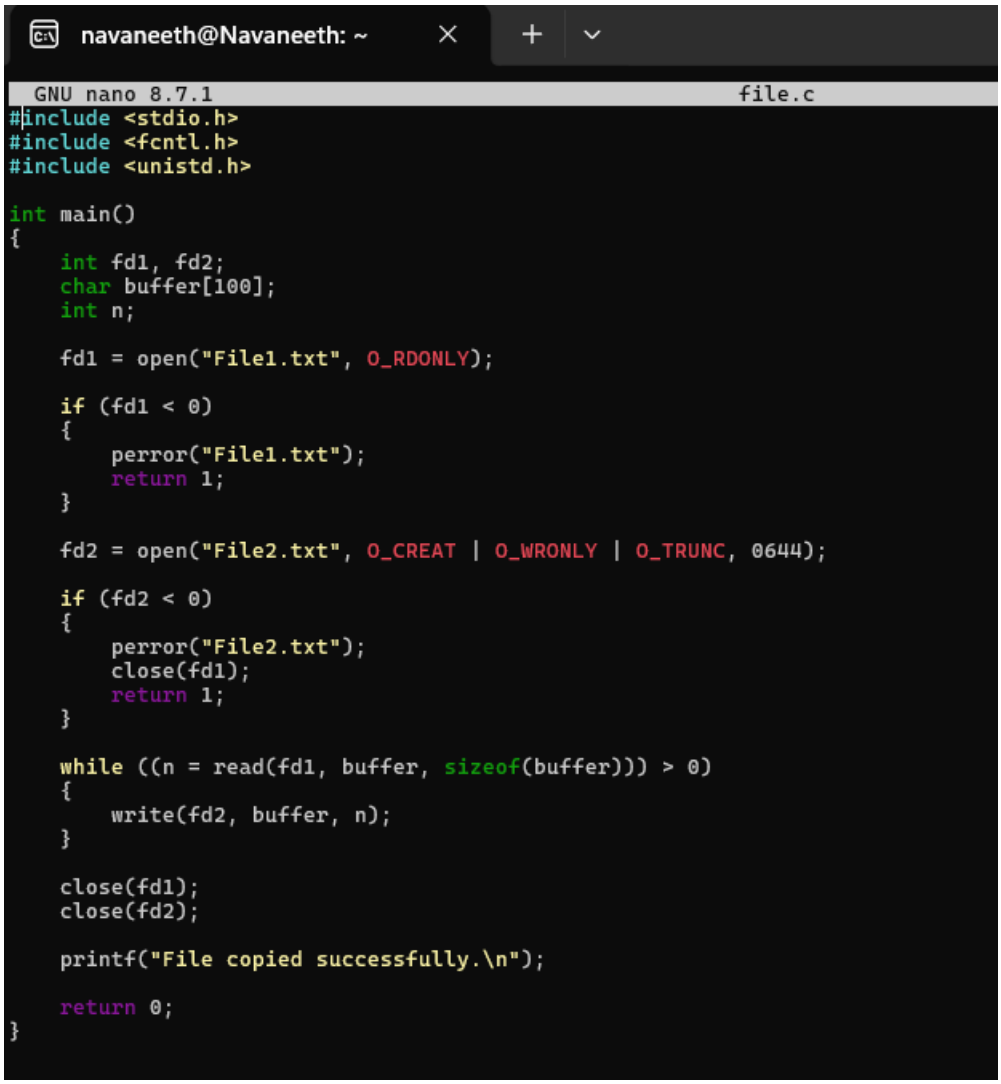
```
navaneeth@Navaneeth:~$ nano shell.c
navaneeth@Navaneeth:~$
navaneeth@Navaneeth:~$ nano shell.c
navaneeth@Navaneeth:~$ gcc shell.c -o shell
navaneeth@Navaneeth:~$ ./shell
myshell>
You entered:
myshell>
You entered:
myshell> 90
You entered: 90
myshell> 30
You entered: 30
myshell>
```

REPORT: The interactive loop was successfully implemented. The program accepts user input repeatedly and terminates when the user enters exit.

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. Open()
2. Read()
3. Write()
4. Close()

CODE:



OUTPUT:

```
navaneeth@Navaneeth:~$ nano file.c
navaneeth@Navaneeth:~$ gcc file.c -o file
navaneeth@Navaneeth:~$ ./file
File1.txt: No such file or directory
navaneeth@Navaneeth:~$
```

REPORT:The program successfully opened File1.txt in read mode and created File2.txt in write mode. The contents of File1.txt were read using read() and written to File2.txt using write(). Finally, both files were closed using close(). Thus, the file was successfully copied using the required system calls.

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support MultiCharacter Commands, Test User Interaction.

CODE:

navaneeth@Navaneeth: ~

GNU nano 8.7.1

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
        printf("No command entered.\n");
    else
        printf("Command: %s\n", input);

    return 0;
}
```

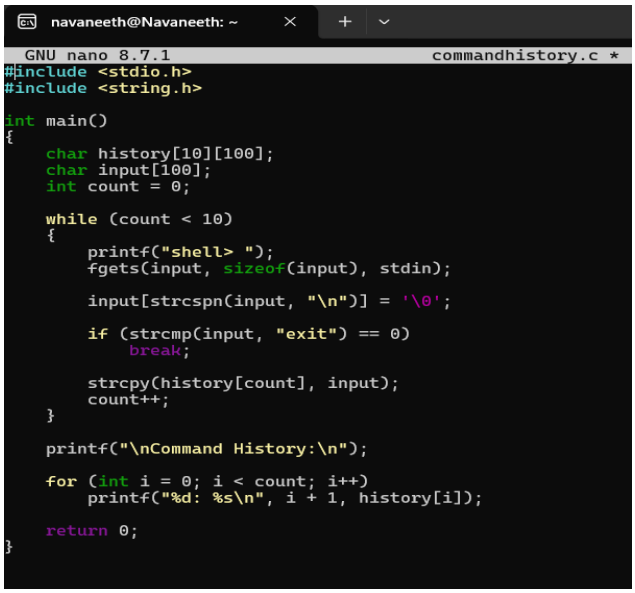
OUTPUT:

```
navaneeth@Navaneeth:~$ nano command.c
navaneeth@Navaneeth:~$ gcc command.c -o command
navaneeth@Navaneeth:~$ ./command
Enter command: ls
Command: ls
navaneeth@Navaneeth:~$
```

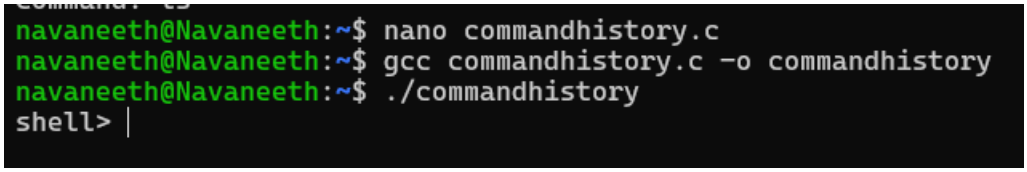
REPORT: The program captures keyboard input, stores it in a buffer, processes the Enter key, and displays the entered command.

Task 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

CODE:



OUTPUT:



REPORT: The command history functionality was successfully implemented. Commands entered by the user were stored and displayed. The program demonstrated basic command recall and input-buffer management as required for the task.

Task 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

CODE:

navanneeth@Navaneeth: ~

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node *head = NULL;
    struct Node *temp;

    for (int i = 1; i <= 3; i++)
    {
        temp = malloc(sizeof(struct Node));

        if (temp == NULL)
        {
            printf("Memory allocation failed.\n");
            return 1;
        }

        temp->data = i;
        temp->next = head;
        head = temp;
    }

    printf("Linked List: ");

    temp = head;

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    temp = head;

    while (temp != NULL)
    {
        struct Node *next = temp->next;
        free(temp);
        temp = next;
    }

    printf("\nMemory released successfully.\n");

    return 0;
}
```

OUTPUT:

navaneeth@Navaneeth: ~

```
navaneeth@Navaneeth:~$ nano MemoryList.c
navaneeth@Navaneeth:~$ gcc MemoryList.c -o MemoryList
navaneeth@Navaneeth:~$ ./MemoryList
Linked List: 3 2 1
Memory released successfully.
navaneeth@Navaneeth:~$
```

REPORT: The program successfully allocated memory dynamically, resized the array, created and managed a linked list, and released the allocated memory using free(). The program can be checked with Valgrind to verify proper memory management and detect memory leaks.